

SWEN90010 2023 Practice Exam - Markdown

Practice Version

Source PDF: materials/2023-pastpaper.pdf

Visible total: 60 marks across 12 questions

Extraction note: the source PDF includes Studocu watermarks and visible submitted-answer annotations. This practice version keeps the exam prompts and omits those submitted answers from the main question text. The base question text was recovered with OCR and checked against rendered page images.

Q1. Fault Trees - 6 marks

A car's automatic stopping system consists of a brake actuator, which is electronically controlled, as well as proximity sensors, plus other components mentioned below. The purpose of the system is to automatically activate the car's brakes when an object is detected as being too close to the front of the car by either of the proximity sensors.

Actuator and ECU. When an object is detected by the proximity sensors as being too close to the front of the car, the system activates a brake actuator which applies the car's brakes. Specifically, an Electronic Control Unit (ECU) receives signals from the proximity sensors. The ECU activates the brake actuator in response to those signals, as long as the ECU has not failed.

Proximity Sensors. The system employs two proximity sensors to detect objects that are too close to the front of the car. Each is a redundant back-up for the other: an object is detected if either of the proximity sensors is triggered, that is, if either of the proximity sensors detects an object that is too close to the front of the car.

Safety condition (the hazard). A hazard occurs when there is an object that is too close to the front of the car but the car's brakes are not applied.

Perform a fault tree analysis on this system for this hazard. Your fault tree should outline how the failure can occur.

The top-level event, i.e. the root, for the fault tree is the case in which there is an object that is too close but the brakes are not applied.

The following components in the system can each fail, and should be assumed to fail independently of one another. These failures should constitute the basic failure events, i.e. the leaf nodes, of your fault tree:

- The proximity sensors, each of which fails independently of the other.
- The ECU.
- The brake actuator.

You may write your fault tree as preformatted text. For instance, to represent a fault tree with root A, child nodes B and C, connected by an OR gate:

```
A
+ OR
  B
  C
```

If node B has children E and F connected by an AND gate:

```
A
+ OR
  B
  + AND
    E
    F
  C
```

Your answer:

Q1 Full-Mark Answer

```
object too close and brakes not applied
+ OR
  brake actuator failed
  brake actuator not applied
+ OR
  ECU failed
  ECU not triggered
+ AND
  proximity sensor 1 failed
  proximity sensor 2 failed
```

Reason: The top hazard occurs if the object is too close but the brakes are not applied. The brakes may fail to be applied because the brake actuator itself fails, or because the actuator is not commanded. The actuator is not commanded if the ECU fails, or if the ECU is not triggered by the proximity sensors. Since either proximity sensor can detect the object, the ECU is not triggered only when both proximity sensors fail.

Schema: Work backwards from the hazard. Alternative sufficient causes use OR; redundant components that must all fail before the function is lost use AND.

Q2. Fault-Tolerant Design - 5 marks

After installing the automatic stopping system described in the previous question, the car manufacturer discovers that the proximity sensors are extremely sensitive, causing many false alarms to be raised, in which the car's brakes are applied even though there is no object that is too close to the front of the car.

It is therefore suggested that the system should be modified so that the brakes are applied only when both proximity sensors detect an object that is too close to the front of the car.

Do you think this change is a good idea? Using your fault-tree analysis or otherwise, explain your answer. If you think the change is a good idea, explain why. If you think it is not a good idea, suggest an alternative design with a brief justification.

Your answer should be no more than about 5 sentences long.

Your answer:

Q2 Full-Mark Answer

No, this is not a good idea for a safety-critical stopping system.
Requiring both sensors to detect the object means that a single failed sensor can prevent braking.
In the fault tree, the sensor-failure cause of "ECU not triggered" changes from "sensor 1 failed AND sensor 2 failed" to "sensor 1 failed OR sensor 2 failed", making the hazard more likely.
A better design is to use three or more sensors with majority voting, or to stop safely when the sensors disagree or no reliable majority is available.
This reduces false alarms without allowing one failed sensor to disable the safety function.

Schema: In safety systems, reducing nuisance activations is not enough. Check whether the proposed change makes the top hazard easier to reach. Changing redundant OR detection into AND detection often turns "both sensors must fail" into "one sensor failure is enough".

Q3. Alloy 1 - 2 marks

Consider a simple Alloy model for part of a hospital records system. The system stores records about patients and about employees. For each employee, the system stores which roles the employee has. There are two roles: `MedicalStaff` and `Administrator`. Each employee can have multiple roles.

For each patient, the system stores which medical staff are assigned to that patient. For each employee, the system stores which other employee, if any, is their supervisor.

We can model this in Alloy with the following signature declarations, where the signature `State` models the state of the records system:

```
sig Patient {}
```

```

sig Employee {}

abstract sig Role {}
one sig MedicalStaff, Administrator extends Role {}

one sig State {
  var roles: Employee -> set Role,
  var supervisor: Employee -> lone Employee,
  var patient_medical_staff: Patient -> set Employee
}

```

The following predicate defines when an Employee *c* is the CEO:

```

pred is_ceo[c : Employee] {
  no State.supervisor[c]
}

```

Explain in words what this predicate is saying. You should explain what notion the predicate is encoding, i.e. what it means for an employee to be the CEO, and how the predicate has encoded that notion.

Your answer:

Q3 Full-Mark Answer

The predicate says that employee *c* is the CEO exactly when *c* has no supervisor recorded in the current State.
 It encodes the notion that the CEO is the employee at the top of the management hierarchy, so no other employee supervises them.
 In Alloy, `State.supervisor[c]` looks up the supervisor of *c* in the supervisor relation, and `'no State.supervisor[c]'` says that this lookup returns the empty set.

Schema: To explain an Alloy predicate in words, state both the domain idea and the relational encoding. Here the domain idea is "CEO has no supervisor"; the encoding is no applied to the supervisor lookup for *c*.

Q4. Alloy 2 - 5 marks

Continuing the hospital example from Q3.

Each state of the system should satisfy the following four conditions:

- The medical staff assigned to each patient must each have the MedicalStaff role.
- All medical staff must have a supervisor.
- An employee cannot be their own supervisor.
- There is exactly one CEO.

Together these conditions define the state invariant of the records system. Write an Alloy predicate `inv` that formally describes when the State satisfies this invariant, i.e. satisfies these conditions.

```

pred inv {
  // Your answer
}

```

Q4 Full-Mark Answer

```

pred inv {
  // The medical staff assigned to each patient must have the MedicalStaff role.
  all p : Patient | all e : State.patient_medical_staff[p] |
    MedicalStaff in State.roles[e]

  // All medical staff must have a supervisor.
  all e : Employee |
    MedicalStaff in State.roles[e] implies some State.supervisor[e]

  // No employee can be their own supervisor.
  all e : Employee |
    e not in State.supervisor[e]

  // There is exactly one CEO.
  one e : Employee | is_ceo[e]
}

```

Schema: Encode each English invariant as one conjunct. For "exactly one employee satisfies predicate P", write one $e : \text{Employee} \mid P[e], \text{not all } e \mid \text{one } P[e]$.

Q5. Alloy 3 - 5 marks

Continuing the hospital example from Q3.

Suppose you are given the implementation of the following function:

```
fun management[e : Employee] : set Employee {  
  // implementation omitted  
}
```

For a given Employee e , $\text{management}[e]$ returns the set that contains e 's supervisor, the supervisor of e 's supervisor, the supervisor of the supervisor of e 's supervisor, and so on. We call this set e 's management.

1. [3 marks] Write and implement an Alloy assertion that asserts that it is always the case that, when the invariant `inv` holds, no employee can be in their own management.
2. [2 marks] Would you expect this assertion to hold? If so, explain why. If not, explain why not and give a counter-example. Your answer for this part should be no more than about 2 sentences.

// Your assertion

Your short explanation:

Q5 Full-Mark Answer

```
assert NoEmployeeInOwnManagement {  
  inv implies  
    all e : Employee | e not in management[e]  
}
```

Equivalent answer:

```
assert management_invariant {  
  all e : Employee | inv implies e not in management[e]  
}
```

This assertion should **not** be expected to hold. The invariant says an employee cannot be their own direct supervisor, but it does not forbid longer management cycles. For example, employee A could be supervised by B, and B could be supervised by A; then neither employee is their own direct supervisor, but each is in their own management chain.

Schema: $e \text{ not in management}[e]$ is an acyclicity property over the transitive supervisor chain. A "no self-supervisor" invariant only rules out cycles of length 1, not cycles of length 2 or more.

Q6. Alloy 4 - 3 marks

Continuing the hospital example from Q3.

Implement the following function which, for a given Employee u , returns the set of Employees for which u is their direct supervisor.

```
fun subordinates[u : Employee] : set Employee {  
  // FILL IN HERE  
}
```

Q6 Full-Mark Answer

```
fun subordinates[u : Employee] : set Employee {  
  { e : Employee | State.supervisor[e] = u }  
}
```

Equivalent relational form:

```
fun subordinates[u : Employee] : set Employee {
```

```

    State.supervisor.u
}

```

Reason: `State.supervisor[e]` is the supervisor of employee `e`. So the subordinates of `u` are exactly the employees `e` whose supervisor is `u`.

Schema: When a relation maps `child -> parent`, finding the parent uses `relation[child]`; finding all children of a parent requires looking backwards through the relation.

Q7. Alloy 5 - 6 marks

Continuing the hospital example from Q3.

Complete the following Alloy predicate that models the operation of changing the hospital's CEO to the new Employee `c`.

All of the old CEO's subordinates become subordinates of the new CEO `c`. After this operation, the old CEO has no subordinates. No other changes to the supervisor relation should occur.

This operation does not modify the roles of any employee. It does not modify the medical staff assigned to each patient. Your definition should preserve the state invariant but, otherwise, should not impose any extra conditions besides those needed to preserve the state invariant and those mentioned above.

```

pred change_ceo[c : Employee] {
  // FILL IN HERE
}

```

Q7 Full-Mark Answer

One suitable answer is:

```

pred change_ceo[c : Employee] {
  one oldCEO : Employee | {
    inv
    is_ceo[oldCEO]
    oldCEO != c

    State.supervisor' =
      (State.supervisor - (c -> Employee))
      ++ ((subordinates[oldCEO] - c + oldCEO) -> c)

    State.roles' = State.roles
    State.patient_medical_staff' = State.patient_medical_staff
    after inv
  }
}

```

Reason: The old CEO is the unique employee with no supervisor in the pre-state. To make `c` the new CEO, `c`'s old supervisor link is removed by subtracting `(c -> Employee)`. The old CEO's former direct subordinates, except `c` itself if necessary, are made subordinates of `c`. The old CEO is also made subordinate to `c`; this is the minimal necessary extra update to avoid having both `oldCEO` and `c` count as CEOs after the transition. The other mutable fields are unchanged, and after `inv` requires the post-state invariant to hold.

If the domain wanted the old CEO to be "retired", the model would need an explicit mutable set such as `activeEmployees` or `retiredEmployees`. In the given model, `Employee` is static and CEO is defined only by having no supervisor, so leaving both `oldCEO` and `c` with no supervisor would violate the "exactly one CEO" invariant.

Schema: For a relation update, write one equation for the whole primed relation. Use relation subtraction to remove old pairs, and `++` to override/add the post-state pairs. Prime the mutable field, not the signature: `State.supervisor'`, not `State'.supervisor`.

Q8. Hoare Logic 1 - 8 marks

Consider the following proof sketch for a program that sums the first `n` elements of an array `A`, where array `A` is indexed from 1, i.e. it is an array `A[1], ..., A[k]` where `k` is the length of array `A`.

Here, n is an arbitrary integer. Text in green in the original PDF is Hoare logic annotations. For an integer i , $\text{SUM}(i)$ is the sum of the first i elements of array A . If i is zero or negative, then $\text{SUM}(i)$ is 0. For this question, we implicitly assume that n does not exceed the length of the array A . This means that you can ignore cases for $\text{SUM}(i)$ and $A[i]$ when i exceeds the length of array A .

The following loop invariant has been used in this proof:

$\text{sum} = \text{SUM}(i) \wedge \text{remaining} + i = n$

where " $\wedge$ " means logical AND.

There is a mistake in the proof. Your task is to identify the mistake and correct it. That is, you should:

1. [2 marks] Explain which step of the proof is incorrect and why.
2. [6 marks] Provide a correct proof sketch. In doing so you are allowed to strengthen the initial precondition, i.e. replace it with one that is stronger than the original. However you are not allowed to modify the final postcondition. You are also allowed to modify the loop invariant, if you wish. If you do so, you should state clearly what is the modified invariant.

```
{true}
{0 = SUM(0) /\ n + 0 = n}
remaining := n
{0 = SUM(0) /\ remaining + 0 = n}
sum := 0
{sum = SUM(0) /\ remaining + 0 = n}
i := 0
{sum = SUM(i) /\ remaining + i = n}
while remaining > 0 do
  {sum = SUM(i) /\ remaining + i = n /\ remaining > 0}
  {sum + A[i+1] = SUM(i+1) /\ remaining - 1 + i + 1 = n}
  i := i + 1;
  {sum + A[i] = SUM(i) /\ remaining - 1 + i = n}
  sum := sum + A[i];
  {sum = SUM(i) /\ remaining - 1 + i = n}
  remaining := remaining - 1;
  {sum = SUM(i) /\ remaining + i = n}
done
{sum = SUM(i) /\ remaining + i = n /\ remaining = 0}
{sum = SUM(n)}
```

Your answer:

Q8 Full-Mark Answer

Part 1: Incorrect step

The incorrect step is the assertion immediately after the loop:

$\{\text{sum} = \text{SUM}(i) \wedge \text{remaining} + i = n \wedge \text{remaining} = 0\}$

At loop exit, the negation of the guard $\text{remaining} > 0$ gives only $\text{remaining} \leq 0$, not $\text{remaining} = 0$.

So the original invariant is too weak to justify the post-loop step. We need an extra fact saying that remaining can never become negative.

Corrected proof sketch:

Modified precondition:
 $\{n \geq 0\}$

Modified loop invariant:
 $\text{sum} = \text{SUM}(i) \wedge \text{remaining} + i = n \wedge \text{remaining} \geq 0$

Proof sketch:

```
{n >= 0}
{0 = SUM(0) /\ n + 0 = n /\ n >= 0}
remaining := n
{0 = SUM(0) /\ remaining + 0 = n /\ remaining >= 0}
sum := 0
{sum = SUM(0) /\ remaining + 0 = n /\ remaining >= 0}
i := 0
{sum = SUM(i) /\ remaining + i = n /\ remaining >= 0}

while remaining > 0 do
  {sum = SUM(i) /\ remaining + i = n /\ remaining >= 0 /\ remaining > 0}
  {sum + A[i + 1] = SUM(i + 1)
   /\ remaining - 1 + i + 1 = n
   /\ remaining - 1 >= 0}
  i := i + 1;
  {sum + A[i] = SUM(i)}
```

```

    /\ remaining - 1 + i = n
    /\ remaining - 1 >= 0}
sum := sum + A[i];
{sum = SUM(i)
  /\ remaining - 1 + i = n
  /\ remaining - 1 >= 0}
remaining := remaining - 1;
{sum = SUM(i) /\ remaining + i = n /\ remaining >= 0}
done

{sum = SUM(i) /\ remaining + i = n /\ remaining >= 0 /\ remaining <= 0}
{sum = SUM(i) /\ remaining + i = n /\ remaining = 0}
{sum = SUM(i) /\ i = n}
{sum = SUM(n)}

```

Reason: The strengthened precondition $n \geq 0$ makes the initial value of `remaining` non-negative. The strengthened invariant $\text{remaining} \geq 0$ is then preserved by the loop because the body only runs when $\text{remaining} > 0$, so after $\text{remaining} := \text{remaining} - 1$, it is still non-negative. At loop exit, the guard is false, so $\text{remaining} \leq 0$. Combining $\text{remaining} \geq 0$ and $\text{remaining} \leq 0$ gives $\text{remaining} = 0$. Then $\text{remaining} + i = n$ gives $i = n$, so $\text{sum} = \text{SUM}(i)$ gives the required postcondition $\text{sum} = \text{SUM}(n)$.

Schema: For a while G do ... done proof, the post-loop fact is invariant $\neg G$. If the final proof needs equality such as $\text{remaining} = 0$, the invariant must contain a lower or upper bound that turns the negated guard into that equality.

Q9. Hoare Logic 2 - 4 marks

Continuing with the array sum example from Q8.

Explain the purpose of the part of the loop invariant that says:

```
remaining + i = n
```

Your answer should be no longer than about four sentences.

Your answer:

Q9 Full-Mark Answer

The condition $\text{remaining} + i = n$ records the relationship between the two loop-control variables and the original input n . The variable i counts how many array elements have already been processed, while remaining counts how many are still left to process. Each loop iteration increases i by 1 and decreases remaining by 1, so their sum stays equal to n . This is needed at loop exit: once $\text{remaining} = 0$, the invariant lets us conclude that $i = n$, so $\text{sum} = \text{SUM}(i)$ becomes the required postcondition $\text{sum} = \text{SUM}(n)$.

Schema: A loop invariant often needs a **linking equation** between a progress variable and a remaining-work variable. Without $\text{remaining} + i = n$, the proof can say "the sum is correct for the first i elements", but it cannot show that i has reached n when the loop finishes.

Q10. Pointers - 7 marks

The following C program takes two pointers p and q , each of which points to an integer, as its arguments. It then creates a local variable t which is also a pointer to an integer. It assigns the value of p to t , i.e. it makes t point to the same location in memory that p points to. It then writes the value that is held in the memory location to which q points into the location to which t points. Finally, it uselessly assigns the value of t to p , i.e. it makes p point to the same memory location to which t points.

```

void prog(int *p, int *q)
{
    int *t;
    t = p;
    *t = *q;
    p = t;
}

```

1. [4 marks] Re-write this program in SPARK Ada. That is, each line should be translated into one-or-more lines of SPARK Ada code that performs exactly the same operations.

You can assume that you already have the following type declaration to represent pointers to integers:

```
type IntPtr is access Integer;
```

That is, you should use the type `IntPtr` to represent pointers to integers in your SPARK program.

2. [3 marks] Explain whether the resulting program satisfies SPARK's aliasing restrictions. Your explanation should be no more than about four sentences.

```
-- Your SPARK Ada translation
```

Your aliasing explanation:

Q10 Full-Mark Answer

```
procedure Prog(P : in out IntPtr; Q : in IntPtr) is
  T : IntPtr;
begin
  T := P;
  T.all := Q.all;
  P := T;
end Prog;
```

If writing a fuller SPARK version, it is also reasonable to include the non-null assumption explicitly:

```
procedure Prog(P : in out IntPtr; Q : in IntPtr)
with Pre => P /= null and Q /= null
is
  T : IntPtr;
begin
  T := P;
  T.all := Q.all;
  P := T;
end Prog;
```

Aliasing explanation: The program satisfies the relevant SPARK aliasing restriction. After `T := P`, `T` and `P` alias, but the program only dereferences `T`; it does not also dereference `P` while the alias is active. The final assignment `P := T` only copies the pointer value back and is effectively a no-op. Therefore the temporary alias does not create an unsafe case where two active aliases are both used to access the same memory.

Schema: `T := P` copies a pointer value; `T.all := Q.all` copies the integer stored at the location pointed to by `Q` into the location pointed to by `T`. Do not confuse pointer assignment with assignment through a pointer.

Q11. Separation Logic - 3 marks

Continuing with the C pointer program from Q10.

Write down a separation logic specification for this program. That is, write down a precondition and a postcondition for this program in separation logic that each mention `p` and `q` and the contents of the memory locations to which they point.

Precondition:

Postcondition:

Q11 Full-Mark Answer

```
Precondition:
p |-> x * q |-> y

Postcondition:
p |-> y * q |-> y
```

Here `x` is the integer initially stored in the cell pointed to by `p`, and `y` is the integer initially stored in the cell pointed to by `q`.

Reason: The command `t = p` only copies the pointer value, so it does not change the heap. The command `*t = *q` writes the old value at `q` into the heap cell pointed to by `p`, because `t` points to the same cell as `p`. The command `p = t` again only copies a pointer value and does not change the heap. The separating conjunction `*` says that `p` and `q` point to separate heap cells, so writing through `p` does not change the cell pointed to by `q`.

Schema: In separation logic, $p \mid \rightarrow x$ describes a heap cell, not a Boolean condition like $p \neq \text{null}$. The separating conjunction $*$ carries the non-aliasing fact that ordinary contracts usually express separately as $p \neq q$.

Q12. SPARK and Design by Contract - 6 marks

Consider the following trivial SPARK Ada procedures.

```
procedure Server(I : in out Integer) with
  Pre => I >= 0,
  Post => I = 0
is
begin
  I := I + 1;
  I := 0;
end Server;

procedure Client(I : in out Integer) with
  Pre => I > 0,
  Post => I >= 0
is
begin
  Server(I);
end Client;
```

Suppose the SPARK Prover were run on this code, to prove that it satisfies the SPARK pre- and post-condition annotations. Explain the checks that the SPARK Prover would perform and what the outcome of each check would be, i.e. whether it would hold and why.

For each check you should describe what property the SPARK prover would check, whether it holds for this example, and why, the reason the check does or does not hold.

Your answer:

Q12 Full-Mark Answer

The SPARK Prover checks that each procedure body satisfies its own postcondition and that each call satisfies the callee's precondition.

1. Server postcondition check

For `Server`, the prover assumes the precondition:

```ada I >= 0 ```

Then it checks that after the body finishes, the postcondition holds:

```ada I = 0 ```

This check holds, because although the body first executes `I := I + 1`, it then executes `I := 0`, so at procedure exit `I` is definitely 0.

2. Client call-precondition check

At the call:

```ada Server(I); ```

the prover must check that `Client` establishes `Server`'s precondition:

```ada I >= 0 ```

This check holds because `Client`'s own precondition is:

```ada I > 0 ```

and `I > 0` implies `I >= 0`.

### 3. Client postcondition check

For Client, after the call to Server( $I$ ), the prover may use Server's postcondition:

```ada I = 0 ```

It must then prove Client's postcondition:

```ada I >= 0 ```

This check holds because  $I = 0$  implies  $I \geq 0$ .

**Runtime-check caveat:** If the prover is also asked to prove absence of runtime errors, then the expression  $I + 1$  in Server may raise an integer overflow when  $I = \text{Integer}'\text{Last}$ . The precondition  $I \geq 0$  is not strong enough to rule that out. However, for the pre/post-condition annotation checks asked for in this question, the three contract checks above all hold.

**Schema:** For a procedure call inside a caller, prove the callee's precondition from the caller's current knowledge; after the call, use the callee's postcondition to prove the caller's remaining obligations.